Proof-Guided Floating-Point Optimization: Exact Reassociation, Error Bounds, and Compiler Certificates

ANONYMOUS AUTHOR(S)

Strict IEEE 754 compilation preserves evaluation order even when the input values make reassociation exact. We use grid and magnitude facts to identify these cases and distinguish bit-identical rewrites from those that preserve a stated error bound or hold only over the reals. For matrix chains, we give sufficient conditions under which every bracketing returns the same correctly rounded result. A separate bound-preservation result applies when overflow and inexact underflow are excluded; an upper magnitude bound alone does not suffice. MatrixLang checks these obligations from declared input domains and exports numerical certificates that a rational-arithmetic checker can replay. ExactReduce derives related facts from integer conversions in LLVM and uses them to guard reduction vectorization or replace floating-point accumulation with integer accumulation. On nine integer-data kernels, the tested Clang pipeline achieved 5.09–9.71× speedups over strict Clang. Licensed and integer-accumulating builds matched strict results on 784 adversarial cases in the local regression run. The pass licensed 23 of 109 reductions in four selected code bases; CPython's `audioop` reached 8.05× speedup. These gains concern integer-valued data, and loop versioning increased compilation time by 29% and text size by 40% in the kernel-module study. The proof covers regrouping the same terms; the LLVM integration remains a tested implementation rather than an end-to-end verified compiler.

CCS Concepts: • **Software and its engineering** → **Compilers**; • **Theory of computation** → *Program analysis*; • **Mathematics of computing** → *Mathematical software.*

Additional Key Words and Phrases: floating-point arithmetic, compiler optimization, fast-math, reassociation, vectorization, abstract interpretation, numerical contracts, matrix chains, LLVM

1 Introduction

Floating-point addition and multiplication are not associative, and $x \cdot 0 = 0$, $x + 0 = x$ and $x \cdot I = x$ fail for signed zeros, NaN and infinities [1, 2]. Compilers therefore forbid such rewrites by default and offer permissions that enable them: GCC's `-ffast-math` and its components [3], or LLVM's per-instruction fast-math flags [4]. A permission states what the compiler *may* do. It does not state what the result *is*: after `-fassociative-math` a programmer cannot tell which outputs still equal the strict program, which changed within a known error bound, and which changed arbitrarily.

The guarantees lead to different optimization decisions. Consider the chain $R = ABC$ with $A, C \in \mathbb{R}^{100\times 2}$ and $B \in \mathbb{R}^{2\times 100}$. Source order costs 69,800 scalar operations; $A(BC)$ costs 1,396. If A, B and C hold 8-bit integers, every product and partial sum of either bracketing is an integer of at most 29 bits, so both bracketings compute the same bits: the rewrite is *bit-identical*, and a strict compiler may apply it. With larger integer values, exactness may fail while the standard componentwise error bound remains valid under every bracketing: the rewrite is *bound-preserving*. A declaration of real values in $[-1, 1]$ alone is insufficient, because it admits inexact underflow. If values are unbounded, one bracketing may overflow where the other does not, and the rewrite is *relaxed*: valid over the reals and nowhere else.

The cases share their program text and differ in the values the program will see (Table 1). We make that knowledge explicit: program inputs declare a value domain (`input(int8)`, `input(int32)`, `input(real(1))`), the runtime checks each input against its domain before execution, and the compiler derives from the domains facts precise enough to decide, rewrite by rewrite, which guarantee holds. Each output is then certified with the weakest guarantee among the rewrites that reach it, and the same facts tell a C compiler which sums it may reorder.

The facts need not come from a new language. Ordinary C states many of them in its types, because integer data is stored as integers and converted where it is summed:

Table 1. One chain $R = ABC$ (40×2, 2×40, 40×2) over four input domains under the repaired admission rule: modeled arithmetic after optimization (11,120 in source order) and the certificate of `print(R)`, as our compiler reports them.

Inputs	strict	bounded	algebraic
`int8`	556, bit-identical	556, bit-identical	556, bit-identical
`int32`	11,120, bit-identical	556, bound-pres.	556, bound-pres.
`real(1)`	11,120, bit-identical	11,120, bit-identical	556, relaxed
`real`	11,120, bit-identical	11,120, bit-identical	556, relaxed

```
double sum(const uint8_t *p, long n) { double s = 0; for (long i = 0; i < n; i++) s += p[i];
    return s; }
```

Under strict semantics Clang keeps this loop scalar, because reordering a floating-point sum may change it; with `-ffast-math` it vectorizes the loop and runs it 6.57× faster on the MNIST training images. Yet every partial sum of every order is an integer below 2^{53} whenever $n \le 2^{53}/255$, so every order returns the same bits. Our LLVM pass proves this from the `uint8_t` conversion, checks n once on entry to the loop, and lets the vectorizer reorder exactly this sum: the strict program runs 6.50× faster and prints the same result.

Contributions.

- Which guarantee a rewrite keeps can be computed per rewrite and composed into per-output certificates, provided a rewrite inherits the level of the facts its proof read (Section 3). Executing 4,800 programs under every contract found the case that makes the inheritance rule necessary; with it, no output certified bit-identical differed (RQ2).
- A matrix chain is bit-identical under every bracketing and summation order when every proper sub-chain fits in the significand and the final sums round at most once (Theorem 4.3), and a reduction with any start value is when a run-time guard holds (Proposition 4.5). The chain threshold is the true boundary on three of four measured families and one step from it on the fourth (RQ3).
- The componentwise error bound of a chain is the same for every bracketing (Theorem 5.1) under explicit range and underflow premises. A grid check discharges the underflow premise; magnitude bounds alone do not. RQ1 measures the opportunities retained by this conservative rule.
- Certificates let a strict compiler take the speed of unproved reordering on integer data: with OpenMP reduction clauses, strict compilation ran integer kernels at 83–113% of that speed, with one result per kernel across every build, compiler and machine (RQ4). ExactReduce, an LLVM pass that derives the facts from integer conversions in C, made nine strict reductions 5.09–9.71× faster with the strict bits (RQ7, RQ8); the same facts let it accumulate those sums in integers, at a median 102% of the speed of hand-written integer code (RQ7). It guards 23 of 109 reductions in four code bases (RQ9).
- Tests of a numerical optimizer need inputs built from its proof obligations: random and extreme-value inputs leave a mutant that admits one more significand bit alive, and inputs placed just past each threshold kill it (RQ5, RQ8).

2 Setting

MatrixLang is a statically shaped, straight-line language over binary64 matrices and scalars. Its compiler (flex, bison, C11) performs shape checking, lowers to three-address code (TAC), optimizes, and executes on a typed stack virtual machine (VM). A matrix product entry is evaluated as

$c_{ij} = (\cdots((+0 + a_{i1}b_{1j}) + a_{i2}b_{2j})\cdots) + a_{in}b_{nj}$: the accumulator starts at +0, each term is one rounded multiplication and one rounded addition in round-to-nearest, and no fused multiply-add is used (`-ffp-contract=off`). Outputs are compared in hexadecimal, which distinguishes finite binary64 values and signed zero, and records NaN classification; it does not distinguish NaN payloads.

The optimizer performs constant folding, common subexpression elimination, copy propagation and dead-code elimination, which are bit-exact; algebraic identity and zero rewrites ($x \cdot 1$, AI, $x + 0$, $A \cdot 0$, $0 \cdot A$); and matrix-chain reordering by dynamic programming over the modeled cost $m\,p\,(2n-1)$ of an $m \times n$ by $n \times p$ product. Before this work a single flag decided all value-changing rewrites at once.

Input domains. A declaration `matrix A[40,2] = input(int8);` introduces a run-time input whose values the compiler never sees. Domains are `bool`, `uint8`, `int8`, `uint16`, `int16`, `int32`, `int(lo,hi)`, `real` (any finite value), `real(B)` ($|x| \le B$) and `real(lo,hi)`. Values arrive from files or a seeded generator and are checked before execution: finite, integral for integer domains, within bounds, and with -0 stored as +0 in integer and non-negative domains. The check is what makes domain-derived facts sound.

LLVM. LLVM attaches fast-math flags to individual instructions [4]: `reassoc` permits reassociation, `nsz` ignores the sign of zero, `nnan` and `ninf` assume no NaN or infinity. The loop vectorizer vectorizes and interleaves an addition reduction, recognized from its header phi, only when every addition of the chain carries `reassoc` or a loop pragma forces it, itself a per-loop license [5]. Otherwise, on targets without ordered vector reductions, the reduction stays scalar, which is why strict semantics cost speed.

3 Contracts and certificates

Definition 1 (Guarantee levels). *A rewrite of program P into P' is* bit-identical *if every output of P' equals the corresponding output of P bit for bit, for every input in the declared domains;* bound-preserving *if every output of P' satisfies the stated standard-model worst-case error bound of P's evaluation order, with that model established for both evaluations, and NaN and infinity occur in P' exactly where they occur in P; and* relaxed *otherwise, when it is an identity over the reals.*

The levels are ordered bit-identical < bound-preserving < relaxed. A *contract* names the weakest level a rewrite may have and still be applied: *strict* (the default) admits only bit-identical rewrites, *bounded* admits bound-preserving ones, and *algebraic* admits all. Each candidate rewrite computes its level from the facts of its operands. The contract applies or declines it and reports a declined rewrite with the contract that would permit it. Constant folding, common subexpressions, copy propagation, dead-code elimination and double transposition are bit-exact and need no proof.

The bounded contract constrains transformations; it does not certify that the unoptimized program itself satisfies a relative-error model. Retaining source order on the underflow counterexample preserves its result, without asserting the inapplicable bound of Theorem 5.1.

Certificates. Every TAC instruction carries the weakest level of the rewrites applied to it. The certificate of an output is the weakest level in the dependency cone of its `print`, together with the reasons recorded by those rewrites. For exact and larger integer-domain chains, the bounded contract reports explanations of the following form (decimal coefficients in the text are rounded for display; the exported hexadecimal coefficient is authoritative):

```
1. bit-identical     print(R)
   - A * B * C is exact: every intermediate of every bracketing needs at most 51 of 53
     significand bits
```

```
2. bound-preserving  print(S)
   - X * B * C keeps the source-order error bound: |error| <= 2.89e-15 |A1|...|Ak|
     under every bracketing; overflow and inexact underflow are excluded
```

Inheritance. A dependency cone in the *optimized* program is not enough. In $R = 0 \cdot (1 \cdot (0 \cdot N)^{\mathsf{T}})$ with $N = -A$, the inner $0 \cdot N$ may be -0; the bounded contract rewrites it to $+0$ as a bound-preserving rewrite. The outer product is then provably $+0$, a bit-identical rewrite, but only because of the inner one, and once the outer rewrite drops every use of the inner product, dead-code elimination removes it and its label. The soundness experiment (Section 11.2) found this case in an earlier version. A rewrite now inherits the weakest level in the cone of the operands its proof read, at the time of the rewrite.

Proposition 3.1 (Bit-identical certificates are sound). *Let optimization apply rewrites $r_1, \ldots, r_n$ in order, each with a level computed from facts that over-approximate its operands' values in the program it rewrites, and let labels follow the inheritance rule. Then every instruction labelled bit-identical computes, for every input in the declared domains, the value of the instruction of the unoptimized program it stands for; in particular every output certified bit-identical prints the unoptimized bits.*

Proof. By induction on k, for the program after $r_1, \ldots, r_k$. Initially every instruction is its own counterpart, and bit-exact passes replace instructions by others that compute the same value from the same operands. Let r_k be bit-identical and replace t by t'. Its proof gives $t' = t$ for every operand value that satisfies the facts it read, and those facts hold in the current program. If every instruction in the cone of those operands is labelled bit-identical, the operands equal their unoptimized counterparts by the induction hypothesis, hence so does t'; otherwise the inheritance rule labels t' weaker and the claim does not apply to it. Rewrites at a weaker level label their results weaker, and dead-code elimination removes instructions but no label of a remaining one, since each label was fixed at its instruction's creation. □

The certificate is a claim over the whole declared domain; executing a program checks it on drawn inputs only (Section 11.2). The proposition proves composition of bit-identity. A weaker output label records the local obligations that reached that output; the label alone is not a numerical error expression for an arbitrary program containing several approximate rewrites.

Replayable numerical obligations. Setting `MATRIXC_CERTIFICATE_JSONL` exports a record for each applied certified chain segment: its dimensions, operand grids, hexadecimal magnitude bounds, finiteness, arithmetic model, guarantee and upward-rounded error coefficient. The separate checker `tools/check_chain_certificate.py` reconstructs the bounds with exact rational arithmetic. It checks every proper sub-chain, every possible root split, the finite-range cap, the underflow condition and the coefficient. A replay over 221 example and generated programs accepted 660 compilations and checked 262 records (166 bit-identical, 96 bound-preserving); three compilations rejected an intentionally invalid example. Tests reject altered grids, enlarged magnitudes and understated coefficients. This removes the compiler's floating-point bound arithmetic from the numerical replay, but still trusts fact extraction, input admission and the connection from a record to emitted code. It is not a proof assistant, a checker for all identity rewrites, or an LLVM translation validator.

4 Exactness from input domains

The grid–magnitude domain. A fact about a matrix X is a tuple $(g, m, \mathit{fin}, \mathit{nn}, \mathit{nz})$: every entry is a multiple of 2^g ($g \geq -1074$), has magnitude at most m, is finite, is non-negative, and is not -0,

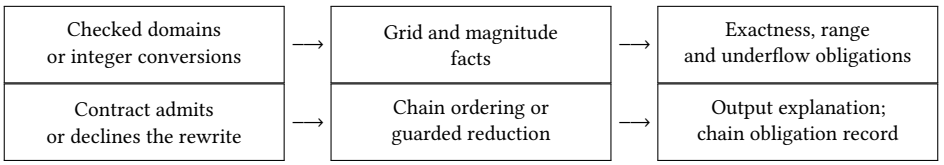


Fig. 1. From value facts to an optimization decision. The obligations in the top row determine admission in the bottom row. MatrixLang exports chain records for independent numerical replay; ExactReduce reports LLVM decisions. Neither report verifies all downstream machine-code transformations.

respectively. Integer domains give $g = 0$ and $m = \max(|lo|, |hi|)$; bounded reals give $g = -1074$ (every binary64 value is a multiple of 2^{-1074}) and $m = B$; unbounded reals give $m =$ DBL_MAX. Literals give exact facts. Transfer functions use upward-rounded bound arithmetic: a sum takes the smaller grid and adds magnitudes; a product of entries adds grids; a matrix product of inner dimension n has grid $g_a + g_b$ and magnitude $n\, m_a m_b$, inflated by $(1 + 2nu)$ when not provably exact. Non-finite or overflowing results lose every fact. A matrix-product entry is never -0, because its accumulator starts at $+0$.

Lemma 4.1 (Rounding preserves grids). *If x is a real multiple of 2^g with $g \geq -1074$ and $|x| \leq$ DBL_MAX, then $\mathrm{fl}(x)$ is a multiple of 2^g, and $\mathrm{fl}(x) = x$ when $|x| \leq 2^{g+53}$.*

Proof. Let $e = \lfloor \log_2 |x| \rfloor$; the spacing of binary64 near x is $2^{\max(e-52,-1074)}$. If $|x| < 2^{g+53}$ then $e - 52 \leq g$, so the spacing divides 2^g and x is representable. If $|x| = 2^{g+53}$, x is a power of two in range. Otherwise $e \geq g + 53$, the spacing 2^{e-52} is a multiple of 2^g, and so is every representable neighbor of x. □

The same holds for binary32 with 24 significand bits and $g \geq -149$.

Lemma 4.2 (Signed zeros of exact sums). *In round-to-nearest, a sum of two multiples of 2^{-1074} is -0 only if both addends are -0. Hence a summation tree whose additions are all exact returns -0 if and only if every leaf is -0, whatever its shape; in particular a tree with a $+0$ leaf never returns -0.*

Proof. An exact zero sum of addends that are not both -0 is $+0$, and a non-zero multiple of 2^{-1074} never rounds to zero. By induction, a subtree is -0 exactly when every leaf is. □

Let $A_1 \cdots A_k$ be a chain with $A_l \in \mathbb{R}^{d_{l-1} \times d_l}$ and facts (g_l, m_l). For $i \leq j$ let $G_{ij} = \sum_{l=i}^{j} g_l$ and $M_{ij} = \prod_{l=i}^{j} m_l \cdot \prod_{l=i}^{j-1} d_l$, which bounds every entry of $|A_i| \cdots |A_j|$. Sub-chain $i..j$ *fits* if $M_{ij} = 0$, or $G_{ij} \geq -1074$ and $M_{ij} \leq \min(2^{G_{ij}+53}, \text{DBL_MAX})$. The cap binds only on coarse grids: for scalars $A = 2^{1023}$, $B = 2$, $C = 0$, the bound of AB is below 2^{G+53}, yet $(AB)C$ is NaN and $A(BC)$ is 0.

Theorem 4.3 (Correctly rounded chains). *Let every operand be finite and every proper sub-chain $i..j \neq 1..k$ fit. Let the whole chain fit, or satisfy $G_{1k} \geq -1074$, $M_{1k} \leq$ DBL_MAX and $(d_s - 1)\, M_{1s} M_{s+1,k} \leq 2^{G_{1k}+53}$ for every split $1 \leq s < k$. Then every bracketing, with the terms of every product entry summed in any order from a $+0$ accumulator, returns $\mathrm{fl}(A_1 \cdots A_k)$ entrywise, and no entry is -0. All such evaluations are bit-identical.*

Proof. Every non-root node T of a bracketing computes its proper sub-chain $P_{ij} = A_i \cdots A_j$ exactly. By induction on $T = LR$ splitting after s: the children are exact, every term $\ell_{ab} r_{bc}$ is a multiple of $2^{G_{is}+G_{s+1,j}} = 2^{G_{ij}}$, and the sum of any subset of the d_s terms has magnitude at most $d_s M_{is} M_{s+1,j} = M_{ij} \leq 2^{G_{ij}+53}$, so by Lemma 4.1 every product and every addition, in any order, is exact. If $M_{ij} = 0$, every term is an exact zero of finite factors.

The root, split after s, has exact children. If the whole chain fits, the root is exact as well. Otherwise every term is at most $M_{1s}M_{s+1,k}$ and every sum of at most $d_s - 1$ terms at most $(d_s - 1)M_{1s}M_{s+1,k} \le 2^{G_{1k}+53}$, so all of them are exact (when $d_s = 1$, the single product is correctly rounded by IEEE 754). In any summation tree, the lowest node whose subtree holds all d_s terms adds two exact partial sums and returns $\mathrm{fl}(c)$ for the exact entry c; any addition above it adds only the accumulator's +0. Lemma 4.2 excludes −0. Every bracketing and every order thus returns $\mathrm{fl}(A_1 \cdots A_k)$. □

This sufficient criterion requires proper sub-chains to be exact: a bracketing can use any of them as an operand, and earlier rounding can change later results. Some chains outside the criterion may still be bit-identical. Checking the whole chain alone is unsound: if the last operand is a zero matrix, $M_{1k} = 0$, yet A_1A_2 may overflow to ∞ and $\infty \cdot 0 = \mathrm{NaN}$ in source order, while the reordered chain computes 0; our mutation study contains this witness (Section 11.5). For `int(0,m)` chains of shape $100 \times 2 \cdot 2 \times 100 \cdot 100 \times 2$ the binding condition is the source order's last product, and the largest admissible bound is $m^\star = 35{,}697$, since $198 \cdot 35697^3 \le 2^{53} < 198 \cdot 35698^3$; requiring the whole chain to be exact stops at 35,578, and Section 11.3 shows that 35,697 is the true boundary.

Corollary 4.4 (Free reductions). *A product LR with inner dimension d, finite operands, $g_L + g_R \ge -1074$ and $(d-1)\, m_L m_R \le \min(2^{g_L+g_R+53}, \mathtt{DBL_MAX})$ gives the same bits for every order of summation of every entry.*

A reduction in C need not start from +0, and its length may be known only at run time. The same argument covers both, and its condition can be checked on entry to a loop.

Proposition 4.5 (Guarded reductions). *Let p be the significand width and $e_{\min}$ the smallest exponent of the format, $G \ge e_{\min}$ with 2^{p+G} finite, and let $s_0, t_1, \ldots, t_N$ be finite multiples of 2^G with $|t_i| \le K2^G$. If $|s_0|/2^G + NK \le 2^p$, then every evaluation of $s_0 + t_1 + \cdots + t_N$ as a summation tree, in any association and order, and with any number of −0 leaves added, returns the exact sum, and returns −0 only when s_0 and every t_i are −0.*

Proof. Every node sums a subset of the leaves: a multiple of 2^G of magnitude at most $|s_0| + NK2^G \le 2^{p+G}$, which is representable (Lemma 4.1). So every addition is exact and every node holds the exact sum of its leaves; added −0 leaves do not change it. Lemma 4.2 gives the sign of a zero result, which depends only on the leaves. □

A vectorized reduction is such a tree: its partial sums start from the identity −0 in every lane but one, which holds s_0.

5 Bound-preserving reassociation

In the standard model $\mathrm{fl}(a \circ b) = (a \circ b)(1 + \delta)$, $|\delta| \le u = 2^{-53}$, absent inexact underflow and overflow, an inner product of length n evaluated in any order satisfies $|\mathrm{fl}(x^{\mathsf{T}}y) - x^{\mathsf{T}}y| \le \gamma_n |x|^{\mathsf{T}}|y|$ with $\gamma_n = nu/(1-nu)$ and $nu < 1$ [7].

Theorem 5.1 (Bracketing invariance). *For every bracketing T of $A_1 \cdots A_k$ for which each contracted dimension satisfies $d_l u < 1$ and every operation obeys the standard model (no overflow or inexact underflow),*

$$|\hat{T} - A_1 \cdots A_k| \;\le\; \Big(\prod_{l=1}^{k-1} (1 + \gamma_{d_l}) - 1 \Big)\, |A_1| \cdots |A_k|$$

componentwise, where $d_1, \ldots, d_{k-1}$ are the inner dimensions.

Proof. Let $\Pi_T = \prod(1 + \gamma_d)$ over the dimensions contracted inside T and $|A|_T$ the product of its operands' absolute values; we show $|\hat{T} - P_T| \le (\Pi_T - 1)|A|_T$ by induction. For $T = LR$ contracting

d, write $\hat{L} = P_L + E_L$, $\hat{R} = P_R + E_R$ and $\hat{T} = \hat{L}\hat{R} + F$ with $|F| \le \gamma_d |\hat{L}||\hat{R}|$. Expanding $\hat{L}\hat{R} - P_L P_R$ and using $|P.| \le |A|.$ and $|\hat{L}| \le \Pi_L |A|_L$ gives $|\hat{T} - P_T| \le (\Pi_L \Pi_R (1+\gamma_d) - 1)|A|_T$. Every inner dimension is contracted at exactly one node of any bracketing, so $\Pi_T = \prod_l (1+\gamma_{d_l})$ for every T. □

The coefficient is independent of bracketing and summation order, conditional on the arithmetic model. The compiler must establish that condition before using the theorem as a rewrite license.

Why an upper bound is insufficient. Let $q = 2^{-1074}$, $a = 3q$, $b = 1/2$ and $c = 3/4$. Round-to-nearest gives

$$\mathrm{fl}(\mathrm{fl}(ab)c) = 2q, \qquad \mathrm{fl}(a\mathrm{fl}(bc)) = q, \qquad abc = 9q/8.$$

The relative errors are $7/9$ and $1/9$, far above the standard-model bound. All inputs satisfy `real(1)`. Embedding them in $A = (a, 0)^\mathsf{T}$, $B = (b, 0)$ and $C = (c, 0)^\mathsf{T}$ gives a 2×1, 1×2, 2×1 chain: the cheaper bracketing costs five modeled operations instead of ten and changes the first output from $2q$ to q. The earlier range-only bounded policy admitted this rewrite. That was a soundness defect, not a case covered by the theorem.

Repaired admission rule. For every nonzero contiguous sub-chain the bounded check now requires $G_{ij} \ge -1074$ and an upward bound $M_{ij} \le 2^{1020}$, together with $d_l u < 1/2$ and $\prod_l (1+\gamma_{d_l}) < 2$. The latter inequalities reserve finite-range slack for rounded intermediate values. By induction, rounded children retain their stated grids (Lemma 4.1); their products and partial sums lie on a representable grid. If a result is subnormal it is therefore exact, and otherwise the relative model applies. The inflated magnitude bounds keep all intermediate products and partial sums finite. Both source and selected trees consequently satisfy Theorem 5.1. Zero sub-chains are exempt from the grid check, but every nonzero proper sub-chain is still checked, including one later multiplied by zero.

This rule is sufficient, not necessary. It retains bounded reassociation for large integer and suitable dyadic domains, while refusing a general `real(1)` chain. The C emitter applies the same product-grid condition before adding a bounded reduction clause. We do not implement an underflow-aware absolute-error contract. All guarantees assume round-to-nearest with gradual underflow, with flush-to-zero and denormals-are-zero disabled. The counterexample and the distinction between exact subnormal products and inexact underflow are executable regressions in the artifact.

6 Segmented chain ordering

A contract may admit a cheaper bracketing for part of a chain but not the whole. For a chain $A_1 \cdots A_k$ and source-order prefixes $P_i = A_1 \cdots A_i$ (computed left to right, as the source does), the pass computes $best[j] = \min_{i<j} best[i] + \mathrm{cost}(P_i \cdot A_{i+1} \cdots A_j)$, where a segment of two operands is the source step, always legal, and a longer segment may use its cheapest bracketing (the classical $O(k^3)$ program [8]) if its level is admitted: bit-identical by Theorem 4.3 for strict, or the repaired range-and-grid test of Section 5 for bounded, anything for algebraic. Segment facts use the facts of P_i as their first operand, and ties prefer the stronger guarantee. Only segment roots carry a guarantee: interior products have no counterpart in the source program, and labelling them let common subexpression elimination spread one chain's proof to an unrelated output. A four-operand `int(0,m)` chain $24 \times 3 \cdot 3 \times 24 \cdot 24 \times 3 \cdot 3 \times 24$ is bit-identical as a whole only for $m \le 2{,}568$ but in its first three operands up to $m = 50{,}727$, and strict reorders that segment (Section 11.3). Operands built in place, such as `ones(n,1)` in $A^3\mathbf{1}$, are moved above the chain so that its products are contiguous.

7 IEEE side conditions of identity rewrites

Table 2 lists the conditions. A matrix product with I is not the identity in IEEE 754: each entry is $+0 + a \cdot 1 + b \cdot 0 + \cdots$, which maps -0 to $+0$ and ∞ to NaN through $\infty \cdot 0$. Adding $+0$ maps -0 to

Table 2. Levels of identity and zero rewrites from operand facts. "fin" is finite, "nz" has no −0 entry, "nn" is non-negative.

Rewrite	bit-identical if	else
$x \cdot 1 \to x$ (scalar unit)	always	–
$AI, IA \to A$ (product)	fin, nz	bounded if fin
$x + 0, 0 + x \to x$	x nz	bounded
$x - 0 \to x$	x nz or zero nz	bounded
$A \cdot 0 \to 0$ (product)	other operand fin	relaxed
$0 \cdot A \to 0$ (scaling)	both nn and fin	bounded if fin

+0. A scaling takes the sign of each factor, so $0 \cdot (-3) = -0$. The rule $0 - x \to -x$ is absent: it maps $+0 - (+0) = +0$ to −0 and saves no arithmetic.

8 Certificates as code

`matrixc --emit-c` writes the optimized program as a C11 translation unit that GCC or Clang compiles like any other. Every instruction becomes a loop nest that performs the VM's arithmetic. A product entry is summed from +0 over k ascending, written i-k-j with literal trip counts so that the C compiler may still vectorize across j, interchange loops, or run rows on separate threads: none of these changes any entry's order of summation. The program reads its inputs from a file, checks them against their domains as the VM does, and prints every output in hexadecimal.

For each product the facts decide the weakest level at which its sums may be reordered: bit-identical by Corollary 4.4; bound-preserving when no partial sum can overflow and no product can incur inexact underflow, since every order keeps the bound $\gamma_d |a|^{\mathsf{T}} |b|$; relaxed otherwise. When the contract admits that level and the output is narrow (at most four columns), the emitter writes the product as dot products under `#pragma omp simd reduction(+:acc)`. OpenMP leaves the order in which such partial results are combined unspecified [6], so the clause is a standard, portable license to reassociate exactly that sum and no other. Compiled without OpenMP, the pragmas are ignored and every sum keeps the VM's order. A comment above each product states its level and the reason.

The backend is also a second implementation of the VM's arithmetic, compiled by a different compiler. On 83 programs (every example and generated programs over all domains), under each contract, GCC's build of the emitted C printed exactly the VM's output, with and without OpenMP SIMD; under strict it printed the *unoptimized* program's output also with every licensed reduction split across vector lanes and four threads (0 mismatches in all). Under bounded and algebraic, licensed reductions changed the output of 27 and 33 programs.

9 The license in LLVM

MatrixLang's facts come from declared domains. C has no such declarations, but its programs state many of the same facts in another way: integer data is stored in integer arrays and converted where it is summed. ExactReduce, an LLVM 18 pass plugin, derives grid facts from those conversions and licenses the reassociation of a reduction only when it proves that every addition tree over the same terms returns the same bits. The plugin registers at LLVM's `VectorizerStart` extension point in the `-O3` pipeline, after the preceding scalar and loop passes, so that subsequent vectorization reads its decisions.

Facts from the IR. A fact (G, K) about a floating-point value states that it is an integer multiple of 2^G of magnitude at most $K2^G$, the grid–magnitude domain of Section 4 with exact integer bounds.

An `sitofp` or `uitofp` of an integer gives G from the integer's known trailing zero bits and K from its range, which LLVM's value tracking and scalar evolution compute (the latter through the loop's guards, so that a loop counter converted to floating point is bounded too); the conversion must itself be exact. Constants give exact facts; `fmul`, `fadd`, `fsub`, `fneg`, `fabs`, `fpext`, `fptrunc`, `select`, phis and `llvm.fmuladd` propagate them as in Section 4, and every operation inside a term must be exact. A value loaded from a floating-point array has no fact: the pass licenses nothing over real-valued data.

Reductions and the static license. The pass takes every floating-point add reduction that LLVM's recurrence analysis recognizes and walks its chain from the loop latch back to the header phi: additions, subtractions of a term from the accumulator, `llvm.fmuladd` with the accumulator as addend, and the selects and phis of conditional reductions. The walk yields W, the largest magnitude one iteration can add. If the start value s_0 has a fact, either a constant or the running sum of enclosing loops that only add this loop's result to it, and scalar evolution bounds the trip count by N, Proposition 4.5 applies when $|s_0| + NW \le 2^{p+G}$; for nested sums N is the product of the trip-count bounds.

The guarded license. When the start value is unknown, or the trip count may be too large, as with any `long` or `size_t` loop bound, the pass versions the loop. The preheader computes

```
a  = fabs(s0);  u = a * 0x1p-G;                  /* exact: G <= 0, scaling by a power of two */
ok = a <= 0x1p(p+G) && u == trunc(u)             /* s0 is a multiple of 2^G, |s0| <= 2^(p+G) */
     && btc < (2^p - (uint64_t)u) / K;           /* |s0| / 2^G + trip count * K <= 2^p      */
if (ok) /* licensed copy of the loop */ else /* the loop as it was */
```

where `btc` is the backedge-taken count. When scalar evolution can express that count only through a division by a loop-invariant value, which it will not compute before the loop because the divisor might be zero, the guard uses the dividend, a valid upper bound. Whenever the guard holds, the condition of Proposition 4.5 holds for this entry into the loop, so the licensed copy prints the bits the original would have. A loop with several guarded reductions gets one copy, guarded by all their conditions. The guard costs a handful of instructions per entry.

Permission and proof boundary. The pass adds `reassoc` to the recognized recurrence instructions, including supported `fsub` and `llvm.fmuladd` forms; it does not add `nsz`, `nnan` or `ninf`. The proposition covers partitioning the same terms among vector lanes and interleaved accumulators, then combining their exact partial sums with -0 identities. It also covers the integer route when the separate accumulator-width checks hold. It does not justify distribution, replacing terms, dropping a signed zero, or other algebraic rewrites merely because they are permitted by `reassoc`. LLVM's flag is a permission, not an enforceable encoding of this proof boundary [4].

The implementation claim is therefore specific to the tested LLVM 18 `clang -O3 -march=native -fpass-plugin=ExactReduce.so` pipeline and its baseline x86-64 variant, using the stated contraction settings. IR regressions check that declined functions acquire no reassociation permission, standalone products acquire none, broader fast-math flags are absent, and guarded functions retain strict fallbacks. Execution tests check the final code. These checks do not prove all LLVM transformations or future releases sound; changing the version, pass order, contraction mode or target requires renewed validation. An end-to-end proof would need translation validation or an explicitly restricted downstream lowering. A versionable loop is innermost, has one exit and has a trip count computable on entry; an unsupported loop keeps its original reduction.

The integer rewrite. Where every term of a licensed sum is an integer, the facts also bound the integer sum, so the pass can accumulate it in an integer register and convert it once after the loop:

the exact sum, hence the strict bits, in lanes narrower than `double`. Terms are rebuilt from their conversions, products, negations and constants. The accumulator has 32 bits when the static bound on the terms' sum, or the guard's limit 2^{p+G}, is below 2^{31}, and 64 bits otherwise; a static license whose terms fit 32 bits is versioned on its trip count so that short entries use 32 bits. The rewrite needs a straight chain, an exit from the latch and a constant start other than -0, the one start for which the strict sum can be -0 (Lemma 4.2); elsewhere the license stays. The study's mutants are switches of the pass: one adds a bit to every significand width, one licenses every reduction without a proof, as `-fassociative-math` does, and one widens the 32-bit limit by two trips.

10 Implementation

The evaluated MatrixLang snapshot contained 7,714 lines of C11, flex and bison, of which the fact domain and chain pass (`facts.c`, `chain.c`) take 1,357 and input domains and their checks (`domain.c`, `inputs.c`) 555; contracts and certificates are in `optimize.c`, the C backend in `emit_c.c`. Flags select the contract (`--fp-strict`, the default, `--fp-bounded`, `--fp-algebraic`), disable facts to reproduce the previous strict optimizer (`--no-proofs`), print certificates and write C; ExactReduce reports each reduction's decision and reason.

Bound arithmetic. Bounds are binary64 values rounded upward, so an understated bound is a soundness defect; an exact-rational oracle found two in the version the measurements ran on. A product bound $p = \mathrm{fl}(ab)$ was raised by one unit when `fma(a,b,-p)` was positive, but near the smallest normal number that residual can fall below the subnormal range and round to zero. The bound then understated the exact product, in 3 of 21 oracle cases with library `fma` and 4 with hardware FMA; the residual is now tested after an exact scaling by 2^{54}. The coefficient of Theorem 5.1 in bounded certificates, formed as $\mathrm{fl}(\prod(1+\gamma_d)) - 1$, understated it for 2 of 85 dimension lists and is now accumulated upward. The first defect needs a bound below 2^{-969}; the second changes only certificate text. After the repair the oracle, now in the test suite, finds no understatement (0), the 174 compilations of the 58 examples under the three contracts print the same certificates, outputs and C (0 differ), and rerunning RQ1–RQ3, RQ5 and RQ6 reproduced 128,015 of their 128,015 records, timings aside. The LLVM pass's bounds are exact integers.

11 Evaluation

We ask: (RQ1) how much of the algebraic optimization each contract recovers with proofs; (RQ2) whether certificates are sound; (RQ3) how close the threshold of Theorem 4.3 is to the true boundary; (RQ4) what certificates are worth to a production compiler, on generated and real data, on two machines and under two C compilers; (RQ5) which tests notice a missing side condition; (RQ6) whether cheaper bracketings are more accurate; and, for the LLVM pass, (RQ7) what its licenses are worth, (RQ8) whether they are sound under inputs built to break them, and (RQ9) how often existing code has reductions they apply to. All scripts, seeds and raw observations are in the artifact. RQ1 and RQ2 are replayed after the underflow repair; the remaining performance and accuracy measurements retain their original snapshots. Their bounded real-domain arms measure the former policy and do not establish a valid bounded guarantee. The strict and LLVM measurements concern the unchanged exactness rules.

Machines. The original RQ1–RQ6 studies and the first half of RQ4 ran on machine M1, an Intel Core Ultra 9 275HX (24 cores, AVX2) under Windows 11 with GCC 15.2.0. The post-repair RQ1 and RQ2 replays ran on M1 under Ubuntu 24.04 in WSL, with GCC 13.3.0, using the same exposed programs and input seeds. They test the repaired implementation rather than provide a new timing comparison. The rest of RQ4 and RQ7–RQ9 ran on machine M2, an Intel Xeon Processor 2.80GHz cloud instance (4 cores, AVX-512) under Linux with GCC 13.3.0 and Clang 18.1.3; the RQ7 baselines

Table 3. RQ1. Modeled arithmetic removed under each contract (pooled over 15,000 programs, ten seeds, six profiles), and recovery: the share of the algebraic contract's saving obtained over the unproved strict optimizer.

Domain	Modeled arithmetic removed (%)				Recovery (%)	
	no proofs	strict	bounded	algebraic	strict	bounded
`bool`	0.4	22.8	23.1	23.1	98.8	100.0
`uint8`	0.5	14.0	17.9	17.9	77.8	100.0
`int8`	0.5	16.5	19.6	19.6	83.6	100.0
`int16`	0.3	7.8	22.3	22.3	34.4	100.0
`int32`	0.3	7.1	19.4	19.4	35.8	100.0
`real(1)`	1.4	7.5	11.8	25.1	25.9	43.7
`real`	0.2	2.5	6.0	27.2	8.4	21.2
`mixed`	0.4	6.4	13.5	16.3	37.9	82.9
`dyadic literals`	0.1	20.2	22.4	22.4	90.2	100.0
`broad literals`	0.2	5.7	20.2	20.2	27.5	100.0

also ran on M1, under WSL with Clang 18.1.3. A speedup is a ratio of medians. The interquartile range of a build's runs was a median 6% of its median on M1 and 11% on M2; the largest, 33% and 117%, occur in builds under 10 ms, and above 10 ms it stays within 20% and 37%. In RQ7 the process medians spread by a median 3% (largest 40%). A ratio near 1.05 is therefore within the noise; the factors above two are not.

Workloads. RQ1, RQ2 and RQ5 use generated straight-line programs whose matrices are inputs over one value domain (`bool` to `int32`, `real(1)`, unbounded `real`, or a per-matrix mixture), with six shape profiles (heterogeneous, square, narrow, balanced, elementwise, and an algebraic profile that adds identity, zero, negation and negative-scaling expressions). Two literal populations reproduce an earlier study: small integers ("dyadic") and values spread over $10^{-12}..10^{12}$ ("broad"). RQ4 uses six kernels written as the mathematics is written, on generated data, and four on real data: the Cora citation graph [42] (2,708 papers, 5,278 undirected citations) and the MNIST digits [43], with an `int8` softmax classifier we trained on its training images (92.22% test accuracy after quantization, 92.19% before). RQ7 and RQ8 use the same data in C kernels; RQ9 uses existing code.

11.1 RQ1: Recovery

Table 3 reports 60,000 compilations, none failing. Without proofs strict optimization removes almost nothing; with proofs it recovers 98.8%, 83.6% and 77.8% of the algebraic saving on `bool`, `int8` and `uint8` with every output bit-identical, and 34.4% and 35.8% on `int16` and `int32`, whose longer chains exhaust the 53-bit budget. The repaired bounded contract retains opportunities where both range and grid obligations hold. The previous claim of complete recovery for all magnitude-bounded domains is withdrawn: `real(1)` alone does not exclude inexact underflow. Its recovery falls to 43.7% on `real(1)` and 82.9% on mixed domains; integer-domain recovery is unchanged. Remaining real-domain savings include identity rewrites and segments with stronger derived facts. Table 3 reports the repaired policy; strict recovers 90.2% on an earlier study's dyadic literals, which were exact all along. Over `int`$(-(2^b-1), 2^b-1)$, strict proves bit-identical 99%, 89%, 46% and 3% of the algebraic contract's chain reorderings at $b = 3$, 7, 15 and 17 (200 programs), and none from $b = 27$, where a product of three operands needs more than 53 bits. Compiled one at a time, 300 of these programs took a median 8.5 ms under strict and 8.5 ms without proofs (median ratio 1.00, largest 1.11), mostly process start-up.

Table 4. RQ2. Outputs of 4,800 executed programs on three input seeds, compared bit for bit with the unoptimized program.

Contract	Outputs	Differ	Bit-identical		Bound-preserving	
			certified	differ	certified	differ
strict	57,600	0	57,600	0	0	0
bounded	57,600	7,097	47,775	0	9,825	7,097
algebraic	57,600	10,815	43,827	0	9,429	6,882

Table 5. RQ3. Largest m for which strict reorders `int(0,m)` chains: under the exact-only condition, by Theorem 4.3 in closed form, and found by binary search on the compiler's certificates; the smallest m with a confirmed witness; strict runs that differed from unoptimized output.

Chain	exact only	Thm. 4.3	compiler	witness	differ
100×2	35,578	35,697	35,697	$m^\star$+1	0
50×4	35,578	35,818	35,818	$m^\star$+1	0
40×8	30,419	30,676	30,676	$m^\star$+1	0
24×3×4	50,013	50,727	50,727	$m^\star$+2	0

11.2 RQ2: Soundness of certificates

Each program ran unoptimized and under every contract on three seeded input draws, 57,600 executions in all (Table 4). Strict output equalled unoptimized output on all 57,600 outputs. Under the bounded and algebraic contracts, 0 and 0 outputs certified bit-identical differed; every output that differed was certified bound-preserving or relaxed. This experiment found the inheritance defect of Section 3: before the fix, 12 outputs per contract, from four programs of the algebraic profile, were certified bit-identical and differed. Execution checks a certificate on drawn inputs only; over the whole domain it rests on Theorem 4.3 and Proposition 3.1. Certificates are conservative: of 3,275 outputs the bounded contract certified below bit-identical, 858 (26.2%) printed the unoptimized bits on all three draws.

11.3 RQ3: Tightness of the threshold

For four chain families over `int(0,m)`, binary search on the compiler's certificates found the largest m for which strict reorders (Table 5). It equals the closed form of Theorem 4.3 in every family, for the four-operand chain that of its first segment. Allowing one final rounding raises every threshold over the exact-only condition, by 0.3–1.4%. We then looked for witnesses, inputs on which the refused bracketing, written out and run unoptimized, prints different bits from the source order, at 12 values of m per family from $m^\star - 3$ to $2m^\star$. Besides generic inputs (all-maximum, six near-maximum and ten seeded draws), a search emulates the VM's arithmetic on one output entry and lowers a few entries of an all-maximum input so that a partial sum before the last is odd and above 2^{53}, and therefore rounds; the VM confirms every witness it finds. No input at or below a threshold separated the bracketings (0 witnesses), and none of the 847 strict runs differed from unoptimized output. For the three three-operand families a witness exists at $m^\star + 1$, so the threshold is the true boundary. For the four-operand chain the first witness is at $m^\star + 2$, and none can exist at $m^\star + 1$: that maximum is even, so an odd partial sum needs a lowered entry in the first operand's row, which brings the sum below 2^{53}. Generic inputs alone found no witness below $1.100\, m^\star$.

Table 6. RQ4 on M1. Six kernels compiled by `matrixc` to C and by GCC 15.2.0 (`-O3 -march=native -ffp-contract=off`): median time of the program as written, and speedups over it of the same C under `-ffast-math`; of the strict contract; of strict with licensed reductions (`-fopenmp-simd`); of the bounded and the algebraic contracts with licensed reductions; and of strict with licensed reductions on eight threads. The algebraic contract reorders without proofs. † marks outputs that differ from the source program. Results counts distinct outputs over eleven builds, including SSE2-only builds.

Kernel	Inputs	Source	Speedup over source						Results
		(ms)	fast-math	strict	+ clauses	bounded	algebraic	8 threads	
`walks`	`bool`	342	1.12	278	288	288	284	1,202	1
`layers`	`int8, uint8`	1.14	0.99	4.98	5.47	5.44	5.69	14.6	1
`image`	`uint8`	4.61	1.02	0.98	1.02	1.05	1.05	1.87	1
`normal`	`int8, int16`	0.0288	1.09	1.27	1.28	1.24	1.25	1.31	1
`lowrank`	`real(1)`	11.8	1.01	1.00	0.93	469†	503†	6.51	2
`diffusion`	`real(0,1)`	325	1.13	1.06	1.05	277†	277†	5.06	2

Historical bounded timings for `lowrank` and `diffusion` used the former range-only policy; they are not speedups licensed by the repaired bounded contract.

11.4 RQ4: Value in a production compiler

Matrix-chain speedups combine fewer scalar operations with changes in the generated loops and parallel execution. They are not vectorization-only gains. RQ7 instead holds each C reduction's input and mathematical sum fixed and measures the effect of the reduction implementation. Table 6 compiles six kernels written as one would write the mathematics: walk counts $A^3\mathbf{1}$ in a 1,024-vertex graph (`bool` adjacency), two stacked quantized layers W_2W_1x, row and column profiles of a $2{,}048^2$ 8-bit image, the normal equations $X^\mathsf{T}X$, $X^\mathsf{T}y$ of an integer-coded regression, and, as real-valued controls, a rank-16 update BAx and three diffusion steps P^3v. Times are medians of 50 runs in five processes. The source build printed the VM's unoptimized output on every kernel.

The large factors come from reordering chains, which changes the amount of work; any chain optimizer that ignores floating point gets them, as the algebraic column shows. What the contracts add is that a strict compiler may take them. Fast-math gains at most 1.13×: GCC reassociates scalar sums but does not reorder matrix chains. The strict contract proves the reorderings of `walks` and `layers` bit-identical and gains 278× and 4.98× with identical bits; with licensed reductions, which the algebraic build also has, 288× and 5.47×, the speed of the unproved reordering (284× and 5.69×). It declines the real-valued chains, which the former bounded policy reordered for 469× and 277×. Those timings are historical measurements with the bounded interpretation withdrawn; the repaired policy declines these chains. Licensed reductions add little on these kernels (0.93–1.10× over strict on one thread). The products they apply to are matrix–vector products, which use each matrix entry once, so data movement rather than the order of additions bounds their speed. Every integer kernel printed one result across all eleven builds, from SSE2 to eight threads, as Theorem 4.3 guarantees. Each real-valued kernel printed two: the source order's, and the reordered chain's, which the bounded and algebraic builds shared on this machine although nothing guarantees it.

Real data. Table 7 repeats the measurement with real inputs: walks of length three on Cora ($A^3\mathbf{1}$, `bool`), three random-walk steps on Cora (P^3v, $P = D^{-1}A$, `real(0,1)`), the normal equations of a fit of the digit label on the central 10×10 pixels of 4,096 MNIST images (`uint8`), and the `int8` classifier's scores summed over 4,096 test images ($WX\mathbf{1}$). Every real input passed its domain check, and the source builds printed the VM's output. Strict proves the walk count bit-identical on the real graph and, with licensed reductions, gains 1,888× under GCC and 1,691× under Clang, against

Table 7. RQ4 on real data, on M2. Four programs over the Cora citation graph and MNIST, columns as in Table 6 (*threads*: strict with licensed reductions on 4 threads), compiled by GCC 13.3.0 and Clang 18.1.3. Results counts distinct outputs over all builds of both compilers.

Kernel	Build	Source	Speedup over source						Results
		(ms)	fast-math	strict	+ clauses	bounded	algebraic	threads	
`cora_walks`	GCC	42,261	0.98	1,168	1,888	2,059	1,816	6,833	1
	Clang	37,463	0.96	946	1,691	1,678	1,837	8,546	
`cora_diffusion`	GCC	41,762	1.00	0.99	0.98	2,130†	2,080†	4.28	6
	Clang	38,707	1.01†	0.95	0.94	1,532†	1,870†	3.97	
`mnist_normal`	GCC	14.8	0.98	1.02	1.10	1.07	0.98	3.30	1
	Clang	16.6	1.11	1.17	1.15	1.19	1.17	2.74	
`mnist_scores`	GCC	25.9	0.87	5.00	9.73	9.34	11.5	61.5	1
	Clang	25.1	1.13	4.86	10.2	8.91	8.74	19.7	

1,816× and 1,837× for the unproved reordering; the classifier's scores gain 9.73× and 10.2×. Each integer program printed one result across all 22 builds. Strict declines the real-valued random walk, which the bounded contract reorders for 2,130× and 1,532× within the source order's error bound, printing a different result. The normal equations contain no chain, so no contract changes them much.

A second machine and compiler. The kernels of Table 6 were rerun on M2 with GCC 13.3.0 and Clang 18.1.3 (the per-kernel table is in the artifact). The source program printed the same bits with all 3 toolchains for 6 of six kernels. Every integer kernel printed a single result across every build on every machine (4 of 4), so the certified outputs are reproducible across vector widths, thread counts, compilers and machines; the real-valued kernels printed up to 7 results each. With licensed reductions, strict compilation ran the integer kernels at 83–113% (median 99%) of the speed of reordering without proofs on both machines and with both compilers. On M2, whose AVX-512 units make reductions cheap once they may be reordered, the clauses matter more: the walk count gains 190× under strict GCC and 563× with them.

Hand-written reductions. The license pays where a reduction is the only parallelism. Six hand-written reductions on integer-valued data, sized so every partial sum is exact, were compiled by GCC strict and with `-fassociative-math`, `-fno-signed-zeros` and `-fno-trapping-math`, which let it reassociate and vectorize them (medians of 7 processes of 21 runs; the per-kernel table is in the artifact). On certified data both builds printed identical bits, and the licensed build was faster by 1.23× (an `int16` dot product, memory-bound) to 6.54× (an `int8` dot product in binary32, compute-bound), and on M2 with GCC 13.3.0 by 1.67–4.89× with identical bits on all 5 certified kernels; the real-valued control changed bits for a comparable speedup. A run-time domain check over both inputs of the `int16` dot product costs 95.1% of the strict kernel, whereas MatrixLang checks once at load time and ExactReduce's guard checks only the start value and the trip count. Under OpenMP, which combines partial sums in an unspecified order, the certified dot product gave one result for every thread count from 1 to 24 and every repetition, and the real-valued one thirteen.

11.5 RQ5: Test adequacy by mutation

Ten mutants each drop one side condition (Table 8). Small-integer programs, the population an earlier differential study relied on, kill no test of mutant 1, which reorders without any proof: by

Table 8. RQ5. Tests that kill each mutant (number of tests per population in parentheses). A test kills a mutant when the mutant's strict-optimized output differs from the correct unoptimized output.

	Mutant	dyadic (90)	broad (90)	domain (720)	extreme (720)	fixtures (15)	witness (7)	boundary (26)
1	reorder without proof	0	51	247	96	2	2	26
2	whole chain only	0	2	4	0	0	1	0
3	54-bit threshold	0	0	0	0	0	0	24
4	*AI*: ignore −0	3	0	9	22	0	1	0
5	*AI*: ignore non-finite	0	0	0	0	0	0	0
6	$A + 0$: ignore −0	1	1	6	21	1	2	0
7	*A*0: ignore non-finite	0	0	0	3	0	2	0
8	0*A*: ignore sign	4	7	45	47	1	1	0
9	negation yields no −0	4	1	15	20	0	3	0
10	one rounding in every sub-chain	0	2	4	0	0	0	0

Table 9. RQ7 on M2. C reductions over integer data, compiled with `-O3 -march=native`: the pass's decision, the median time under strict Clang 18.1.3, and speedups over it of strict Clang with ExactReduce, of Clang `-ffast-math`, of ExactReduce for baseline x86-64 (SSE2), and of GCC 13.3.0 strict and `-ffast-math`. Medians of five processes of eleven runs. † marks outputs that differ from strict Clang's. Below the line, the two controls.

Kernel	Data	License	Strict	Speedup over strict Clang				
			(ms)	licensed	fast-math	lic. SSE2	GCC	GCC f.-m.
sum	MNIST	guarded	59.1	6.50	6.57	2.91	0.99	4.53
sum of squares	MNIST	guarded	59.4	6.31	6.21	2.57	0.96	4.63
Gram matrix	MNIST	static	386	5.42	5.33	1.51	0.98	3.08
int8 classifier	MNIST	guarded	93.9	9.71	10.2	3.98	0.97	6.71
degrees	Cora	static	9.11	9.19	9.25	2.91	1.01	5.95
2-walks	Cora	guarded	9.31	5.69	5.77	1.82	0.99	3.12
GEMM	generated	static	69.3	5.09	5.35	1.35	1.00	3.02
energy	generated	guarded	5.33	9.15	9.39	3.44	0.98	6.45
sum	generated	guarded	2.67	8.36	8.75	3.81	0.99	6.43
16-term sums	MNIST	guarded	23.4	1.65	1.68	0.93	0.66	1.32
dot product	generated	declined	3.81	0.97	1.54†	1.04†	1.01†	1.43†
sum, binary32	MNIST	guarded	59.9	1.02	8.48†	1.00	1.01	7.32†

Theorem 4.3 they are bit-identical under every bracketing, so the population is blind to the defect; broad-valued literals kill it in most programs. Random programs rarely exercise the side conditions on −0 and non-finite values; one witness program per condition, written from Table 2, kills mutants 1, 2, 4 and 6–9. Mutant 5 is equivalent: a non-finite fact never carries "no −0", so the −0 condition already blocks every rewrite it would admit. Mutant 2, which checks only the whole chain, and mutant 10, which lets every sub-chain round once, are each killed by a few random programs (2 broad-valued and 4 domain tests for mutant 2, 2 and 4 for mutant 10). Mutant 3, which accepts 54 bits, survives every population drawn at random or at the domain's bounds, all-maximum inputs included; only the boundary population, RQ3's witnesses placed just above each threshold, kills it (24 of 26 tests).

The exploratory accuracy analysis is reported in Appendix A. It measures an association between cost and error, not a causal mechanism.

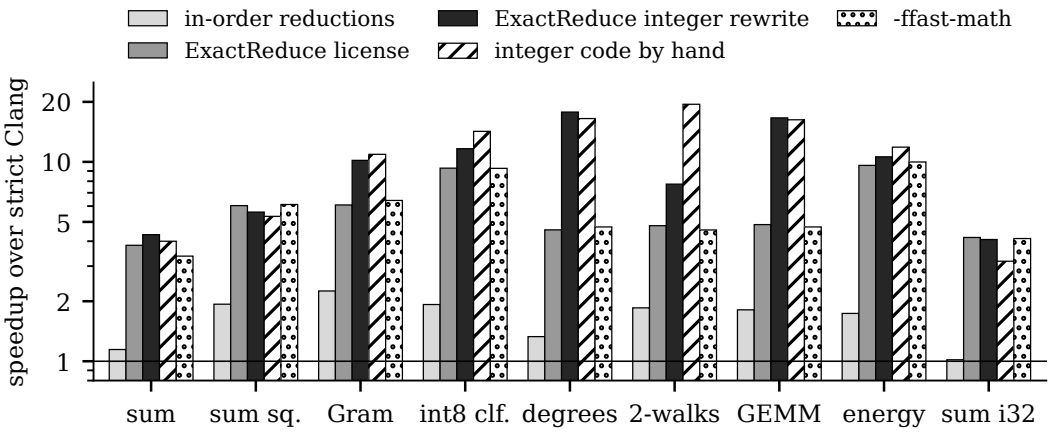


Fig. 2. RQ7 on M1: every way to speed up the nine integer reductions while keeping their bits, and `-ffast-math`, as speedups over strict Clang 18.1.3 (log scale, one run). In-order vector reductions keep the source order; the license lets the vectorizer reorder; the integer rewrite accumulates in integers; hatched bars are the kernels rewritten by hand with integer accumulators. Every bar printed the strict bits.

11.6 RQ7: The license in LLVM

Table 9 compiles eleven kernels written as C programs write them, with integers in integer arrays: the sum and sum of squares of the 47 million `uint8` MNIST training pixels, the Gram matrix of their central 10×10 pixels over 60,000 images, the `int8` classifier's binary32 scores on the 10,000 test images, degrees and 2-walks on Cora, an `int8` GEMM, the energy of `int16` samples and a sum of `int32` values, with two controls: a real-valued dot product and a binary32 pixel sum, exact only up to 65,793 terms. ExactReduce licensed 3 reductions statically, 7 behind a guard, and declined the real-valued one.

With the license, strict Clang ran the nine integer kernels 5.09–9.71× faster, at 95–102% of the speedup of `-ffast-math`, and every one printed the strict bits. So did `-ffast-math`, because these sums are exact in every order; it reassociates without knowing that. On the real-valued control it changed the result (1.54×), and on the binary32 sum, whose guard sends the 47 million terms to the strict loop, it ran 8.48× faster with a different sum, while the licensed build ran at the strict speed (1.02×) with the strict bits. With SSE2 only, the license still gained 1.35–3.98×. GCC's strict builds ran at Clang's strict speed and GCC's `-ffast-math` 3.02–6.71×. The row *16-term sums* exposes the guard's cost by calling the sum kernel once per run of 16 pixels, so that every call evaluates a guard for 16 iterations: the licensed build ran at 1.65× the strict speed and `-ffast-math` at 1.68×.

Compilation and code-size costs. For the kernel translation unit, compilation took a median 178 ms with the pass and 138 ms without (29% more), and loop versioning grew text size from 4,149 to 5,801 bytes (40%). These are module-level measurements over ten compilations; the artifact does not retain per-function compile-time attribution. The existing-code build totals in RQ9 describe a different workload and should not be read as contradicting this kernel-module overhead.

All routes to the same bits. Figure 2 times every build that keeps the strict bits: in-order vector reductions (`-force-ordered-reductions`), the license, the integer rewrite, and the kernels rewritten by hand with the narrowest safe integer accumulator. Over strict Clang, in-order reductions gained 1.02–2.25× (median 1.81), the license 3.82–9.6× (median 4.85), and the integer rewrite 4.08–17.8× (median 10.2), a median 102% of the hand-written code's speed (40–128%). It falls short where the

Table 10. RQ8 on M2. Stress cases whose output differs from strict Clang's, per build: 784 cases over 11 kernels, 7 input families and trip counts up to twice each guard's limit, split into integer kernels and the real-valued control. Strict builds disagree with each other on the control because Clang contracts $s + ab$ into a fused multiply-add where the target has one (its default `-ffp-contract=on`).

Build	integer (707)	real (77)
Clang + ExactReduce	0	0
Clang + ExactReduce, SSE2	0	35
GCC, strict	0	35
Clang `-ffast-math`	14	22
GCC `-ffast-math`	14	42
mutant: one bit more	10	0
mutant: no proof	14	22

Table 11. RQ9 on M2. Floating-point add reductions that LLVM recognizes in four existing code bases, compiled with `clang -O3 -march=native` and ExactReduce, and why the declined ones were declined: no grid fact (real-valued data), a term or sum that is never exact, or a loop the pass cannot version.

Code base	Files	Add reductions			Declined: why		
		static	guarded	declined	no fact	inexact	shape
CPython `audioop`	1	0	16	2	0	1	1
Pillow `libImaging`	68	0	7	15	15	0	0
SciPy `ndimage`	7 of 9	0	0	39	39	0	0
PolyBench/C	30	0	0	30	30	0	0

programmer knows a tighter bound than the types: the 2-walks degrees are below 169, but the pass sees `int16`. All 36 builds of the integer kernels printed the strict bits.

11.7 RQ8: Soundness of the LLVM license

Random inputs rarely break an unsound reassociation (RQ5): sums of random integers stay far below 2^{53}. The stress suite therefore runs every kernel on families built from the proof obligation: all-maximum and all-minimum values, alternating signs that cancel, products that are -0, sparse maxima, and odd maxima, whose partial sums are odd and round as soon as they exceed the significand; and at trip counts that include each guard's limit, one past it, and twice it. Table 10 compares every build with strict Clang. The licensed build printed the strict bits in all 784 cases, with AVX-512 and with SSE2 alike on the integer kernels. `-ffast-math` changed 14 integer cases, all at trip counts past a guard's limit, where the strict order rounds (0 at or below one). The mutant that admits one more significand bit changed 10, all past the true limit and at most twice it (0 at or below it): the condition of Proposition 4.5, like the threshold of RQ3, cannot be relaxed by a bit. The mutant that licenses without proof behaved as `-ffast-math`. On M1 the license and the integer rewrite changed none of the cases, and at GEMM's 32-bit limit the rewrite matched strict on all 20 boundary cases, where a mutant with a limit two terms larger changed 4; in-order reductions changed 20 real-valued cases, forming no fused multiply-add where scalar strict Clang does.

11.8 RQ9: Existing code

Table 11 compiles the C sources of CPython's `audioop` module, now the `audioop-lts` package; Pillow's imaging core; SciPy's `ndimage` [44]; and the 30 kernels of PolyBench/C 4.2.1 [45]. Of 109 reductions, the pass licensed 23, all behind a guard, because every one needed a run-time length or

start value checked. All are sums of integer samples or pixels: `audioop`'s average, root mean square, peak-to-peak average and the 16-bit correlation behind `findfit`, `findmax` and `findfactor`, and Pillow's reduction of 32-bit images. It declined 84 reductions over real-valued data, including all 30 in PolyBench, 1 whose terms are never exact (the root mean square of 32-bit samples squares values up to 2^{31}), and 1 in loops it cannot version. The pass changed the build time of the 106 files from 27.8 to 27.9 s.

End to end, `audioop` was compiled with and without the pass, each build was loaded in its own interpreter, and every function with a reduction was called on fragments of every sample width, lengths from 0 to $2^{22} + 1$ samples (both sides of the 16-bit guard's limit) and the families of RQ8: 404 of 404 results were identical. `findfit` on a 2,000-sample reference ran 5.78× faster, and `avg` and `rms` of 8-bit samples 8.05× and 6.55×. For 16-bit samples they gained nothing (0.96×, 1.05×): their loops advance by the run-time sample width, and the vectorizer speculates that this stride is one byte, so wider samples run the scalar loop. Existing code thus has reductions that the license speeds up without changing a result, but fewer than kernels written for the study: most floating-point reductions in scientific code sum floating-point data.

11.9 Threats to validity

Internal. Certificates and licenses were tested on drawn and constructed inputs only. The theorems justify the specified numerical rewrites under their premises; they do not verify the full implementations. RQ3, RQ5 and RQ8 test boundaries where a wrong condition would become observable. The two implementations share no code but share their authors' reading of IEEE 754; the C backend checks the VM's arithmetic with a second compiler, and the LLVM study compares against Clang's own strict build. *External.* The MatrixLang workloads are generated or written by us, the real data are two public data sets, and RQ9's code bases were chosen because they compile without their build systems, not at random; RQ9 counts reductions LLVM recognizes after inlining. *Construct.* Modeled arithmetic is a cost model; runtime was measured in RQ4, RQ7 and RQ9 on two machines, one a shared cloud instance whose noise medians over processes reduce but do not remove.

12 Related work

Fast-math semantics. GCC's `-fassociative-math` requires `-fno-signed-zeros` and `-fno-trapping-math` [3]; LLVM attaches `reassoc`, `nsz`, `nnan` and other permissions to individual instructions [4]. Both state what may change rather than what holds. Icing gives fast-math style optimizations a semantics in a verified compiler [9], and RealCake verifies them against a real-valued semantics with a proved error bound [10]; there the program opts in and the bound is the guarantee. Our levels are derived: a strict program gets reassociation exactly where it is provably bit-identical, and ExactReduce grants LLVM's permission only there. Alive and Alive-FP verify peephole optimizations with floating-point preconditions on signed zeros and non-finite values [11, 12], Alive-Infer infers such preconditions [13], and Alive2 validates LLVM translations [14]; our side conditions (Table 2) are of this kind, organised into guarantee levels, discharged from domain facts and reported per output. Boldo et al. verify that CompCert preserves floating-point semantics exactly [15]. Monniaux catalogues why compiled floating-point code is hard to reason about [16], among them that reassociation breaks compensated summation [17], which our licenses never touch because such sums are not exact.

Value facts in production compilers. GCC's ranger tracks floating-point ranges with whether a value may be NaN and the sign of its zeros [18], and LLVM's `KnownFPClass`, since LLVM 17, whether a value may be NaN, infinite or −0 [19]; both can discharge side conditions like those of Table 2 for single instructions. LLVM folds floating-point arithmetic on integer-to-float conversions

into integer arithmetic when both are exact [20], a one-operation instance of the integer rewrite. These facilities supply useful local facts. Our contribution combines grid and magnitude reasoning with chain obligations, guarded reduction permissions and per-output explanations; it is not a new theory of rounding. The loop vectorizer versions loops on aliasing and stride conditions [5], ExactReduce on the condition of Proposition 4.5. Siso et al. evaluate vectorizing compilers by withdrawing the information they rely on [27]; the license is such information, derived rather than supplied.

Accuracy analysis and repair. Rosa and Daisy compile real-valued specifications to finite precision under error bounds [21, 22], FPTaylor bounds round-off [23], Herbie rewrites expressions for accuracy [24], and Precimonious [28] and Angerd et al. [25] lower precision where a test or a quality threshold still passes. Chen and Singh convert floating-point code to fixed point from value profiles [26], which holds for the profiled inputs; the integer rewrite converts only sums proved exact for every input. CIGEN finds input ranges on which optimization levels change results [29]; we prove, for declared ranges, which rewrites cannot.

Exactness and reproducibility. The Ozaki scheme splits floating-point matrices into slices whose products are exact [30] and emulates binary64 GEMM on integer units [31, 32]; FFLAS-FFPACK keeps intermediates below 2^{53} to do exact integer linear algebra on floating-point units [33]. Theorem 4.3 turns the arithmetic fact these libraries rest on into a compiler legality criterion over chains of any bracketing, with signed zeros, sub-chains and one final rounding. Error-free transformations make sums exact or accurate [34], reproducible summation makes results independent of order at a cost [35, 36], oneMKL reproduces results for a fixed thread count and code path [37], and FPRev shows that libraries' accumulation orders differ [38]; where our certificates apply, the tested thread counts, vector widths, compilers and machines produce the same results without a reproducible-summation algorithm (RQ4, RQ8). The guards and code growth still have the costs reported in RQ7. Theorem 5.1 follows from the standard analysis of inner products [7]; we claim its use as a contract.

Matrix chains and testing. Optimal chain ordering is classical [8] and Linnea generates linear-algebra code with property-aware cost models [39]; neither certifies guarantees per output. Differential testing [40] and mutation analysis [41] are established; RQ5 and RQ8 show why value-blind differential testing misses reassociation defects.

13 Limitations

MatrixLang is straight-line and statically shaped; loops would need facts that reach a fixed point, which the lattice permits but MatrixLang does not implement. ExactReduce handles loops, but only reductions of integer conversions in loops it can bound or version, and its integer rewrite uses the bounds the types and trip counts imply: where the programmer knows tighter ones, as for the 2-walks degrees, hand-written integer code is faster (RQ7). The bounded admission rule now excludes inexact underflow. It declines general bounded-real chains unless stronger grid facts are available, and provides no absolute-error guarantee for underflowing executions. Real-valued data have grid 2^{-1074}, so bit-identity proofs apply to integer and dyadic data. Domains are trusted declarations checked at load time; a domain wider than the data loses opportunities, never soundness. The threshold is the true boundary for the families of RQ3, not in general: magnitude bounds cannot see, for instance, that every input is even. The proofs are on paper. The independent rational checker validates exported chain obligations conditional on supplied facts; neither the fact analysis nor the complete LLVM pipeline is mechanically verified. Runtime was measured on two machines with three compiler versions; targets with ordered vector reductions may value the license differently. The equal-cost ablation in RQ6 cannot separate two mechanisms.

14 Conclusion

A floating-point rewrite keeps one of three guarantees, and the values a program will see often prove which: a strict compiler can then take the bit-identical reorderings, and a bounded one those that keep an established error bound. The underflow counterexample shows why input magnitude alone cannot establish the latter guarantee. LLVM already has the facts for integer data converted where it is summed: licensing exactly those reductions gives strict C programs 95–102% of the speedup of `-ffast-math` with the strict bits, and accumulating them in integers a median 102% of hand-written integer code. Execution found an inheritance defect, an exact-rational oracle two understated bounds, and only proof-derived inputs killed the one-bit mutants.

Data availability

The local artifact holds the compilers, the LLVM pass, scripts and raw observations. The review-revision directory retains the counterexample checks, IR regressions and post-repair corpus replays separately from historical measurements. The accompanying reproduction instructions identify the scripts that regenerate each display. Reviewer access must be arranged with submission.

Use of generative AI

Generative AI tools assisted with implementation, literature search, evaluation scripts, analysis and drafting, including this revision. Scientific claims, artifact validation and final submission approval remain the authors' responsibility.

A Exploratory accuracy of cheaper bracketings (RQ6)

Table 12. RQ6. Paired comparison of the cost-optimal and source bracketings on real-valued chains, and the equal-cost ablation. Sign tests are two-sided and exact; Holm-adjusted over the three confirmatory tests; median ratio of errors (cost-optimal / source) with seed-cluster bootstrap 95% intervals.

Population	chains	unequal	cheaper better	p (Holm)	median ratio [95% CI]
heterogeneous	648	635	411	3.1×10^{-13}	0.892 [0.863, 0.913]
narrow	606	587	317	0.058	0.979 [0.954, 1.000]
pooled	1,254	1,222	728	4.6×10^{-11}	0.941 [0.925, 0.955]
Equal-cost ablation: first = fewer intermediate entries, or enumeration order					
heterogeneous	64	59	27	0.6	
narrow	363	352	199	0.016	
square	900	885	426	0.28	
pooled	1,327	1,296	652	0.85	

Under its arithmetic premises, Theorem 5.1 gives every bracketing the same worst-case bound; it says nothing about typical error. The protocol was recorded in the artifact, not a public registry, before the confirmatory run: chains of three to five real-valued operands with dimensions up to 200, values uniform in $[-1, 1]$, uniform in $[0, 1]$, or broad; ten new seeds; error $\max_{ij} |\hat{c}_{ij} - c_{ij}| / (|A_1| \cdots |A_k|)_{ij} / u$ against an exact integer reference; and a numpy emulation of the VM's evaluation order, which matched `matrixc` bit for bit on 120 of 120 validation bracketings. It predicted that the pooled sign test rejects equality at 0.01 for the cost-optimal order, in the same direction in every profile, and that an equal-cost ablation removes the effect.

The cost-optimal bracketing, the one of least modeled arithmetic, was more accurate in 728 of 1,222 chains with unequal error (Holm $p = 4.6 \times 10^{-11}$; median ratio 0.941, 95% CI [0.925, 0.955]), significantly on heterogeneous shapes (411 of 635, $p = 3.1 \times 10^{-13}$, ratio 0.892) and in the same

direction but not significantly on narrow shapes (317 of 587, $p = 0.058$), where cheaper bracketings save less. Pairs of distinct bracketings with equal modeled cost showed no difference (652 of 1,296, $p = 0.85$), as an operation-count account predicts. Because equal-cost pairs almost always also have equal intermediate sizes, the ablation does not separate that account from the rival that smaller intermediates cause the effect; we report the association, not the mechanism. No chain exceeded the bound of Theorem 5.1; the largest error reached 0.491 of it.

References

[1] David Goldberg. 1991. What Every Computer Scientist Should Know About Floating-Point Arithmetic. *ACM Comput. Surv.* 23, 1 (1991), 5–48. https://doi.org/10.1145/103162.103163

[2] IEEE. 2019. *IEEE Standard for Floating-Point Arithmetic*. IEEE Std 754-2019.

[3] Free Software Foundation. GCC Manual: Options That Control Optimization (`-fassociative-math`, `-fno-signed-zeros`, `-fno-trapping-math`). https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html

[4] LLVM Project. LLVM Language Reference Manual: Fast-Math Flags. https://llvm.org/docs/LangRef.html#fast-math-flags

[5] LLVM Project. Auto-Vectorization in LLVM. https://llvm.org/docs/Vectorizers.html

[6] OpenMP Architecture Review Board. 2021. *OpenMP Application Programming Interface, Version 5.2*, Section 5.5.6, Reduction Scoping Clauses. https://www.openmp.org/spec-html/5.2/openmpsu50.html

[7] Nicholas J. Higham. 2002. *Accuracy and Stability of Numerical Algorithms* (2nd ed.). SIAM.

[8] T. C. Hu and M. T. Shing. 1982. Computation of Matrix Chain Products. Part I. *SIAM J. Comput.* 11, 2 (1982), 362–373.

[9] Heiko Becker, Eva Darulova, Magnus O. Myreen, and Zachary Tatlock. 2019. Icing: Supporting Fast-Math Style Optimizations in a Verified Compiler. In *CAV 2019*, 155–173. https://doi.org/10.1007/978-3-030-25543-5_10

[10] Heiko Becker, Robert Rabe, Eva Darulova, Magnus O. Myreen, Zachary Tatlock, Ramana Kumar, Yong Kiam Tan, and Anthony Fox. 2022. Verified Compilation and Optimization of Floating-Point Programs in CakeML. In *ECOOP 2022*, LIPIcs 222, 1:1–1:28. https://doi.org/10.4230/LIPIcs.ECOOP.2022.1

[11] Nuno P. Lopes, David Menendez, Santosh Nagarakatte, and John Regehr. 2015. Provably Correct Peephole Optimizations with Alive. In *PLDI*.

[12] David Menendez, Santosh Nagarakatte, and Aarti Gupta. 2016. Alive-FP: Automated Verification of Floating Point Based Peephole Optimizations in LLVM. In *SAS*.

[13] David Menendez and Santosh Nagarakatte. 2017. Alive-Infer: Data-Driven Precondition Inference for Peephole Optimizations in LLVM. In *PLDI*.

[14] Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. 2021. Alive2: Bounded Translation Validation for LLVM. In *PLDI*, 65–79. https://doi.org/10.1145/3453483.3454030

[15] Sylvie Boldo, Jacques-Henri Jourdan, Xavier Leroy, and Guillaume Melquiond. 2015. Verified Compilation of Floating-Point Computations. *J. Autom. Reason.* 54, 2 (2015), 135–163.

[16] David Monniaux. 2008. The Pitfalls of Verifying Floating-Point Computations. *ACM Trans. Program. Lang. Syst.* 30, 3 (2008).

[17] W. Kahan. 1965. Further Remarks on Reducing Truncation Errors. *Commun. ACM* 8, 1 (1965), 40. https://doi.org/10.1145/363707.363723

[18] Aldy Hernandez. 2023. How We Expanded GCC Value Range Propagation for Floats. Red Hat Developer. https://developers.redhat.com/articles/2023/01/18/how-we-expanded-gcc-value-range-propagation-floats

[19] LLVM Project. 2023. `computeKnownFPClass` and `KnownFPClass`, release 17. https://github.com/llvm/llvm-project/blob/release/17.x/llvm/include/llvm/Analysis/ValueTracking.h

[20] LLVM Project. 2024. InstCombine `foldFBinOpOfIntCasts`, release 18. https://github.com/llvm/llvm-project/tree/release/18.x/llvm/lib/Transforms/InstCombine

[21] Eva Darulova and Viktor Kuncak. 2014. Sound Compilation of Reals. In *POPL*.

[22] Eva Darulova, Anastasiia Izycheva, Fariha Nasir, Fabian Ritter, Heiko Becker, and Robert Bastian. 2018. Daisy: Framework for Analysis and Optimization of Numerical Programs. In *TACAS*.

[23] Alexey Solovyev, Charles Jacobsen, Zvonimir Rakamarić, and Ganesh Gopalakrishnan. 2015. Rigorous Estimation of Floating-Point Round-off Errors with Symbolic Taylor Expansions. In *FM*.

[24] Pavel Panchekha, Alex Sanchez-Stern, James R. Wilcox, and Zachary Tatlock. 2015. Automatically Improving Accuracy for Floating Point Expressions. In *PLDI*.

[25] Alexandra Angerd, Erik Sintorn, and Per Stenström. 2017. A Framework for Automated and Controlled Floating-Point Accuracy Reduction in Graphics Applications on GPUs. *ACM Trans. Archit. Code Optim.* 14, 4 (2017), 1–25. https://doi.org/10.1145/3151032

[26] Doris Chen and Deshanand P. Singh. 2013. Profile-Guided Floating- to Fixed-Point Conversion for Hybrid FPGA-Processor Applications. *ACM Trans. Archit. Code Optim.* 9, 4 (2013), 1–25. https://doi.org/10.1145/2400682.2400702

[27] Sergi Siso, Wesley Armour, and Jeyan Thiyagalingam. 2019. Evaluating Auto-Vectorizing Compilers through Objective Withdrawal of Useful Information. *ACM Trans. Archit. Code Optim.* 16, 4 (2019), 1–23. https://doi.org/10.1145/3356842

[28] Cindy Rubio-González, Cuong Nguyen, Hong Diep Nguyen, James Demmel, William Kahan, Koushik Sen, David H. Bailey, Costin Iancu, and David Hough. 2013. Precimonious: Tuning Assistant for Floating-Point Precision. In *SC '13*. https://doi.org/10.1145/2503210.2503296

[29] Dolores Miao, Ignacio Laguna, and Cindy Rubio-González. 2024. Input Range Generation for Compiler-Induced Numerical Inconsistencies. In *ICS '24*, 201–212. https://doi.org/10.1145/3650200.3656618

[30] Katsuhisa Ozaki, Takeshi Ogita, Shin'ichi Oishi, and Siegfried M. Rump. 2012. Error-Free Transformations of Matrix Multiplication by Using Fast Routines of Matrix Multiplication and Its Applications. *Numer. Algorithms* 59, 1 (2012), 95–118. https://doi.org/10.1007/s11075-011-9478-1

[31] Hiroyuki Ootomo, Katsuhisa Ozaki, and Rio Yokota. 2024. DGEMM on Integer Matrix Multiplication Unit. *Int. J. High Perform. Comput. Appl.* 38, 4 (2024), 297–313. https://doi.org/10.1177/10943420241239588

[32] Katsuhisa Ozaki, Yuki Uchino, and Toshiyuki Imamura. 2026. Ozaki Scheme II: A GEMM-Oriented Emulation of Floating-Point Matrix Multiplication Using an Integer Modular Technique. *Int. J. High Perform. Comput. Appl.* https://doi.org/10.1177/10943420261467787

[33] Jean-Guillaume Dumas, Pascal Giorgi, and Clément Pernet. 2008. Dense Linear Algebra over Word-Size Prime Fields: the FFLAS and FFPACK Packages. *ACM Trans. Math. Softw.* 35, 3 (2008).

[34] Takeshi Ogita, Siegfried M. Rump, and Shin'ichi Oishi. 2005. Accurate Sum and Dot Product. *SIAM J. Sci. Comput.* 26, 6 (2005).

[35] James Demmel and Hong Diep Nguyen. 2013. Fast Reproducible Floating-Point Summation. In *ARITH*, 163–172.

[36] Willow Ahrens, James Demmel, and Hong Diep Nguyen. 2020. Algorithms for Efficient Reproducible Floating Point Summation. *ACM Trans. Math. Softw.* 46, 3 (2020). https://doi.org/10.1145/3389360

[37] Intel. 2025. oneMKL Developer Guide: Obtaining Numerically Reproducible Results. https://www.intel.com/content/www/us/en/docs/onemkl/developer-guide-linux/2025-2/obtaining-numerically-reproducible-results.html

[38] Peichen Xie, Yanjie Gao, Yang Wang, and Jilong Xue. 2025. Revealing Floating-Point Accumulation Orders in Software/Hardware Implementations. In *USENIX ATC '25*, 1425–1440.

[39] Henrik Barthels, Christos Psarras, and Paolo Bientinesi. 2021. Linnea: Automatic Generation of Efficient Linear Algebra Programs. *ACM Trans. Math. Softw.* 47, 3 (2021). https://doi.org/10.1145/3446632

[40] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and Understanding Bugs in C Compilers. In *PLDI*. https://doi.org/10.1145/1993316.1993532

[41] Yue Jia and Mark Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Trans. Softw. Eng.* 37, 5 (2011).

[42] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Gallagher, and Tina Eliassi-Rad. 2008. Collective Classification in Network Data. *AI Magazine* 29, 3 (2008), 93–106. https://doi.org/10.1609/aimag.v29i3.2157

[43] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-Based Learning Applied to Document Recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324. https://doi.org/10.1109/5.726791

[44] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17 (2020), 261–272. https://doi.org/10.1038/s41592-019-0686-2

[45] Louis-Noël Pouchet and Tomofumi Yuki. 2016. PolyBench/C 4.2.1. https://sourceforge.net/projects/polybench/